Lesson 8 Teacher's Key: Under Pressure

TIC-80 - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Adding a second piece of state (lives) that changes inside an existing loop's existing branch, alongside a first (enemies_remaining) that changes in a completely different loop - reinforcing the two-separate-loops structure introduced in Lesson 7.

Level 1 - Full Worked Solution

- lives=3 is declared near the top, parallel to enemies_remaining - same pattern, different variable, different loop responsible for changing it.
- The lives decrement is added directly inside the EXISTING escape branch of the enemy-drawing loop (where e.alive=false already happens for reaching the bottom) - one new line in an already-correct branch.
- The Game Over print() check at the bottom of TIC() mirrors the existing You Win! check exactly, just testing lives instead of enemies_remaining.

Highlighted lines are new this week; everything else is code you already have.

```
lives=3

-- enemy movement/escape (drawing loop, revised):
for enemy_index=1,#enemies do
 local enemy=enemies[enemy_index]
 if enemy.alive then
  enemy.ey=enemy.ey+0.3
  if enemy.ey>136 then
   enemy.alive=false
   lives=lives-1
  else
   spr(3,enemy.ex,enemy.ey,0)
  end
 end
end

-- bottom of TIC(), alongside the You Win! check:
if lives<=0 then
 print("Game Over",40,60,8)
end
```

Why This Structure

- This is the direct payoff of Lesson 7's two-loop structure: the enemy-drawing loop was already the single, unambiguous home for "this enemy reached the bottom," so lives could be added there with zero risk of touching the separate bullets-vs-enemies collision loop.
- Worth explicitly re-stating the tradeoff from Lesson 7 here: keeping the two loops separate (rather than merging them) is exactly what made this week's addition a one-line change instead of a careful untangling.

Level 2 / Level 3 Talking Points

- Level 3 sound, worked (TIC-80): import both .wav files into two SFX slots (e.g. 1 and 2). In the enemy-drawing loop's escape branch, after lives=lives-1: if lives<=0 then (if not has_lost then has_lost=true; sfx(2) end) else sfx(1) end. The has_lost guard stops the game-over sound replaying every frame.
- Ask advanced students to trace, out loud, which loop owns which variable now - the enemy-drawing loop owns lives, the bullets loop owns enemies_remaining - as a way of testing whether the Lesson 7 structure actually stuck.
- Level 3 structure: this is a good moment to ask whether TIC-80's cart is starting to feel like it needs more organization (functions instead of one long TIC()) - a preview of concerns that become more pressing as the capstone grows through Lesson 11.

Level 2 - complete code:

```
-- Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
-- Checks: shooting an enemy never changes lives (only the kill path
changes enemies_remaining);
-- an escape never changes enemies_remaining (only the escape path changes
lives);
-- "Game Over" appears once, when lives first reaches 0.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
lives=3
has_lost=false

-- enemy-drawing loop (Level 3):
for enemy_index=1,#enemies do
 local enemy=enemies[enemy_index]
 if enemy.alive then
  enemy.ey=enemy.ey+0.3
  if enemy.ey>136 then
   enemy.alive=false
   lives=lives-1
   if lives<=0 then
    if not has_lost then
     has_lost=true
     sfx(2)
    end
   else
    sfx(1)
   end
  else
```

```
   spr(3,enemy.ex,enemy.ey,0)
  end
 end
end

-- bottom of TIC() (Level 3 visual readout):
print("Lives: "..lives,4,4,12)
if lives<=0 then
 print("Game Over",40,60,8)
end
-- sfx slots: 1 = sfx_lifelost.wav, 2 = sfx_gameover.wav
```

Common Student Mistakes

- Adding the lives decrement inside the bullets loop instead of the enemy-drawing loop (makes shooting cost a life, which is backwards).
- Forgetting lives=3 is declared before TIC() runs (a global, not something reset each frame) - a common Lua scoping confusion worth checking directly if lives behaves oddly.
- "Game Over" and "You Win!" print() calls overlapping on screen because both use the same Y coordinate.